CPSC 250 – Programming for Data Manipulation
Final Exam Solutions

Part I: Multiple Choice

1. C
2. B
3. C
4. C
5. A
6. B
7. A
8. B
9. B
10. C
11. A
12. B
13. B
14. A
15. B

Part II: Error Identification and Correction

One corrected version is shown below.

```
def above_average(values):
   total = 0
   for value in values:
      total += value

   average = total / len(values)

   result = []
   for value in values:
      if value > average:
         result.append(value)

   return result

measurements = [4.2, 5.1, 3.8, 6.0, 4.9]
print(above_average(measurements))
```

Important errors include:

- The loop should accumulate the values using `total += values[i]` or `total += value`; the original line overwrites `total` each time.
- The `if` statement needs a colon.
- `append` is a method and should be called with parentheses, not brackets.
- The variable name in the final line should be `measurements`, not `measurement`.

Part III: Code Writing

Q1. Functions and collections of objects

One possible solution is:

```
def average_height_with_sunlight(plants, min_sunlight):
   total = 0
   count = 0

   for plant in plants:
      if plant.sunlight >= min_sunlight:
         total += plant.height
         count += 1
```

```
    if count == 0:
        return 0

    return total / count
```

A solution using a temporary list is also fine:

```
def average_height_with_sunlight(plants, min_sunlight):
    heights = []

    for plant in plants:
        if plant.sunlight >= min_sunlight:
            heights.append(plant.height)

    if len(heights) == 0:
        return 0

    return sum(heights) / len(heights)
```

Q2. Classes and collections of objects

```
class Book:

    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    def __str__(self):
        return f"{self.title} by {self.author} ({self.pages} pages)"

    def is_long(self):
        return self.pages > 300


for book in library:
    if book.is_long():
        print(book)
```

Q3. Inheritance and polymorphism

```
class Account:

    def __init__(self, owner, balance):
```

```
        self.owner = owner
        self.balance = balance

    def monthly_update(self):
        return self.balance


class SavingsAccount(Account):

    def __init__(self, owner, balance, rate):
        super().__init__(owner, balance)
        self.rate = rate

    def monthly_update(self):
        self.balance = self.balance * (1 + self.rate)
        return self.balance


accounts = [
    Account("Alice", 1000),
    SavingsAccount("Bob", 1000, 0.01)
]

for account in accounts:
    print(account.monthly_update())
```

The loop demonstrates polymorphism because it calls `monthly_update()` on each object without needing to know whether the object is an `Account` or a `SavingsAccount`.

Q4. pandas, plotting, and model functions

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("experiment.csv")

valid = df[df["valid"] == True]

plt.scatter(valid["time"], valid["position"])
plt.xlabel("time")
plt.ylabel("position")

def linear(x, m, b):
    return m*x + b
```

It would also be acceptable to write the filter as:

```
valid = df[df["valid"]]
```

Part IV: Code Comprehension and Commenting

Possible comments are shown below. Student answers do not need to match this wording exactly, but they should explain the purpose of each section or function.

```
# Import the libraries needed for data analysis and plotting.

import pandas as pd
import matplotlib.pyplot as plt

# Define a class to represent a plant in the data set.

class Plant:

   # Initialize a new Plant object with its attributes.
   def __init__(self, name, height, sunlight):
      self.name = name
      self.height = height
      self.sunlight = sunlight

   # Return a readable string description of the plant.
   def __str__(self):
      return f"{self.name}: {self.height} cm, {self.sunlight} hours"

   # Determine whether the plant needs additional attention.
   def needs_attention(self):
      return self.height < 10 or self.sunlight < 4


# Read the CSV file and create a list of Plant objects.

plants = []

df = pd.read_csv("plants.csv")

for _, row in df.iterrows():
   plants.append(
      Plant(row["Name"], row["Height"], row["Sunlight"])
   )
```

```
# Select and print the plants that need attention.

attention = [p for p in plants if p.needs_attention()]

for p in attention:
   print(p)


# Create a DataFrame and make a bar chart of plant heights.

plot_df = pd.DataFrame({
   "Name": [p.name for p in plants],
   "Height": [p.height for p in plants]
})

plt.bar(plot_df["Name"], plot_df["Height"])
plt.xlabel("Plant")
plt.ylabel("Height (cm)")
plt.title("Plant Heights")
plt.show()
```

Questions

1. After the `for` loop completes, the `plants` list stores `Plant` objects.
2. The program is an example of using a collection of objects because it stores many `Plant` objects in a list and then processes those objects using loops and list comprehensions.
3. A plant is added to the `attention` list when `p.needs_attention()` returns `True`. In this program, that means the plant's height is less than 10 or its sunlight value is less than 4.
4. The list comprehensions near the end extract the `name` and `height` attributes from each `Plant` object so that those values can be placed into a pandas DataFrame for plotting.
5. The main loop could still work because of polymorphism. If a `Flower` object inherits from `Plant` and provides its own version of `needs_attention()`, then the code can still call `p.needs_attention()` without needing major changes.

Part V: Short Answer / Conceptual Design

1. Choosing the right representation is important because the representation determines what operations are natural, efficient, and easy to express. For example, a raw CSV

file is useful for storage, a pandas DataFrame is useful for filtering and aggregating data, a list of objects is useful when each item has related attributes and behaviors, and a plot or model can summarize patterns. Good data representation makes the algorithm simpler and makes the program easier to understand, test, and extend.

2. Refactoring into functions and classes makes a program easier to test because each function or method can be checked separately. It also makes debugging easier because the source of an error is usually confined to a smaller piece of code. Classes help organize related data and behavior, while functions reduce repeated code. This makes the program easier to extend because a change can often be made in one place rather than copied into many repeated blocks.